

Estimating Estimands with Estimators

Jake Bowers — EGAP Learning Days, Santiago de Chile

2026-07-30

Table of contents I

- 1 Key points
- 2 Review
- 3 Estimands and estimators and averages
- 4 Block randomization
- 5 Cluster randomization
- 6 Binary outcomes
- 7 Covariance adjustment
- 8 Conclusion
- 9 Causal effects that differ by groups or covariates
- 10 Causal effects when we do not control the dose

Section 1

Key points

Key points about estimation I

- A causal effect, τ_i , is a comparison of unobserved potential outcomes for each unit i , for example:
$$\tau_i = Y_i(T_i = 1) - Y_i(T_i = 0) \text{ or } \tau_i = \frac{Y_i(T_i=1)}{Y_i(T_i=0)}.$$
- To learn about τ_i , we can treat τ_i as an **estimand** or a target quantity to be estimated (discussed here), or as a target quantity to be hypothesized about (session on hypothesis testing).
- Many people focus on the **estimand** of the **average treatment effect** (ATE), $\bar{\tau} = \frac{1}{n} \sum_{i=1}^n \tau_i$, in part, because it allows easy **estimation**.

Key points about estimation II

The key to estimation in causal inference is to choose an estimand that helps you learn about some theoretical or policy question. The ATE is one option, but other common estimands include the ITT, LATE/CACE, ATT, or the ATE for some subgroup (or even a difference of a causal effect between groups).

- An **estimator** is a recipe for calculating a guess (a number or an interval) about the value of an estimand. For example, the difference of observed means for m treated units is one estimator of $\bar{\tau}$:

$$\hat{\bar{\tau}} = \frac{\sum_{i=1}^n (T_i Y_i)}{m} - \frac{\sum_{i=1}^n ((1-T_i) Y_i)}{(n-m)}.$$

This is something we can calculate with observed data. We call such a calculation an “estimate” because we hope that this calculation with observed quantities tells us something about something unobservable like: $\bar{\tau} = \bar{Y}_i(1) - \bar{Y}_i(0)$.

Key points about estimation III (Tests and confidence intervals)

- The **standard error** of an estimator in a randomized experiment summarizes how the estimates would vary if the experiment were repeated. (It does not commonly refer to the variability of samples from a population.)
- We use the **standard error** to produce **confidence intervals** and **p-values** so that we can begin with an estimator and end with a hypothesis test.
- Different randomizations will produce different values of the same estimator targeting the same estimand. A **standard error** summarizes this variability in an estimator.
- A $100(1 - \alpha)\%$ **confidence interval** is a collection of hypotheses that cannot be rejected at the α level. It is common to report confidence intervals containing hypotheses about values of our estimand and to use our estimator as a test statistic.

Key points about estimation IV

- Estimators should have a known relationship to “the truth” (the estimand):
 - avoid systematic error in estimating the estimand (be unbiased);
 - vary little in their estimates from one experiment to another (be precise or efficient); and
 - perhaps ideally converge to the estimand as they use more and more information (be consistent).

Being “close” to the estimand (precise with little or no bias)

Key points about estimation V

- **Analyze as you randomize** in the context of estimation means that (1) our standard errors should measure the variability arising from the randomization and (2) our estimators should target estimands defined in terms of potential outcomes.
- We do not say that we use **control variables** when we analyze data from randomized experiments although we do use **covariance adjustment** (which is the identical calculation). Covariates can make our estimation more **precise**. Note that controlling for covariates in observational studies differs from **covariance adjustment** in randomized experiments even though both use the same `lm(Y~Z+x)` command in R.

Section 2

Review

Review: Causal effects

Review: Causal inference can be summarized as a comparison of unobserved, fixed, potential outcomes.

For example:

- the potential, or possible, outcome for unit i when assigned to treatment, $T_i = 1$ is $Y_i(T_i = 1)$.
- the potential, or possible, outcome for unit i when assigned to control, $T_i = 0$ is $Y_i(T_i = 0)$

Treatment assignment, T_i , has a causal effect for unit i , that we call τ_i , if $Y_i(T_i = 1) - Y_i(T_i = 0) \neq 0$ or $Y_i(T_i = 1) \neq Y_i(T_i = 0)$.

Section 3

Estimands and estimators and averages

How can we learn about causal effects using observed data?

- 1 Recall: we can **test hypotheses** about the pair of potential outcomes $\{Y_i(T_i = 1), Y_i(T_i = 0)\}$.
- 2 We can **define estimands** in terms of $\{Y_i(T_i = 1), Y_i(T_i = 0)\}$ or τ_i , **develop estimators** for those estimands, and then calculate the values and standard errors for those estimators.

A common estimand and estimator: The average treatment effect and the difference of means

Say we are interested in the ATE, or $\bar{\tau} = \frac{1}{n} \sum_{i=1}^n \tau_i$. What would be a good estimator?

Two candidates:

- 1 The difference of means: $\hat{\tau} = \frac{\sum_{i=1}^n (T_i Y_i)}{m} - \frac{\sum_{i=1}^n ((1-T_i) Y_i)}{n-m}$.
- 2 A difference of means after top-coding the highest Y_i observation (a kind of “winsorized” mean, which aims to prevent extreme values from exerting too much influence over our estimator; used to increase *precision*).

How would we know which estimator is best for a particular research design?

Let's simulate!

Simulation step 1: create some data with a known ATE

Notice that we need to *know* the potential outcomes and the treatment assignment in order to learn whether our proposed estimator does a good job.

Z	y0	y1
0	0	10
0	0	30
0	0	200
0	1	91
1	1	11
1	3	23
0	4	34
0	5	45
1	190	280
1	200	220

The true ATE is 54

In real life we can observe only one realization of the potential outcomes. Recall that each unit has its own treatment effect.

First: make fake data

The table on the previous slide was generated in R with:

```
# We have 10 units
N <- 10
# y0 is the potential outcome under control
y0 <- c(0, 0, 0, 1, 1, 3, 4, 5, 190, 1000)
# Each unit has its own intrinsic treatment effect
tau <- c(10, 30, 200, 90, 10, 20, 30, 40, 90, 20)
## y1 is the potential outcome under treatment
y1 <- y0 + tau
# Two blocks: a and b
block <- c("a", "a", "a", "a", "a", "a", "b", "b", "b", "b")
# Z is the treatment assignment
# (we use Z instead of T in the code)
Z <- c(0, 0, 0, 0, 1, 1, 0, 0, 1, 1)
# Y is the observed outcome
Y <- Z * y1 + (1 - Z) * y0
# The data
dat <- data.frame(Z = Z, y0 = y0, y1 = y1, tau = tau, b = block, Y = Y, id = seq_len(N))
set.seed(12345)
```

DeclareDesign

DeclareDesign represents research designs in a few steps:

```
# # Take just the potential outcomes under control and treatment
small_dat <- dat[, c("y0", "y1", "id")]

# The first step in DeclareDesign is to declare the population
pop <- declare_population(small_dat)
N <- nrow(pop)

# 2 units assigned to treatment; DD does complete assignment with m=2
trt_assign <- declare_assignment(Z = conduct_ra(N = N, m = 2))
# Observed Y is y1 if Z=1 and y0 if Z=0
pot_out <- declare_potential_outcomes(Y ~ Z * y1 + (1 - Z) * y0)

# Specify outcome variable and treatment assignment
reveal <- declare_reveal(Y, Z)

# The basic research design object
# includes these four objects
base_design <- pop + trt_assign + pot_out + reveal
```


DeclareDesign: make fake data

DeclareDesign renames y0 and y1 to Y_Z_0 and Y_Z_1 by default:

```
## A simulation is one random assignment of treatment
sim_dat1 <- draw_data(base_design)

# Simulated data (just the first 6 lines)
head(sim_dat1)
```

	y0	y1	id	Z	Y_Z_0	Y_Z_1	Y
1	0	10	1	0	0	10	0
2	0	30	2	1	0	30	30
3	0	200	3	0	0	200	0
4	1	91	4	0	1	91	1
5	1	11	5	0	1	11	1
6	3	23	6	1	3	23	23

Using DeclareDesign: define estimands and estimators

No output here. The code just defines functions, estimators, and one estimand.

```
## The estimand
estimandATE <- declare_inquiry(ATE = mean(Y_Z_1 - Y_Z_0))

## The first estimator is the difference of means
diff_means <- declare_estimator(Y ~ Z,
  inquiry = estimandATE,
  .method = lm_robust, se_type = "classical", label = "Diff-Means/OLS"
)
```

DeclareDesign: define estimands and estimators

```
## The second estimator is the top-coded difference of means

## First make a function to top-code the maximum value
topcode_var <- function(x) {
  rank_x <- rank(x)
  ## Replace the maximum value of Y with the second-highest value of Y
  new_x <- ifelse(rank_x == max(rank_x), Y[rank_x == (max(rank_x) - 1)], Y)
  return(new_x)
}

## Second, write a DeclareDesign-style estimator
diff_means_topcoded_fn <- function(data) {
  data$newY <- topcode_var(data$Y)
  obj <- lm_robust(newY ~ Z, data = data, se_type = "classical")
  res <- tidy(obj) %>% filter(term == "Z")
  return(res)
}

diff_means_topcoded <- declare_estimator(
  handler = label_estimator(diff_means_topcoded_fn),
  inquiry = estimandATE, label = "Top-coded Diff Means"
)
```

DeclareDesign: define estimands and estimators

Here we show how the DD estimators work using our simulated data.

```
## Demonstrate that the estimand works:
```

```
estimandATE(sim_dat1)
```

```
inquiry estimand
```

```
1      ATE      54
```

```
## Demonstrate that the estimators estimate
```

```
## Estimator 1 (difference of means)
```

```
diff_means(sim_dat1)[-c(1, 2, 10, 11)]
```

```
estimate std.error statistic p.value conf.low conf.high df
```

```
1   -123.6    258.6   -0.4781  0.6454   -719.9    472.7   8
```

```
## Estimator 2 (top-coded difference of means)
```

```
diff_means_topcoded(sim_dat1)[-c(1, 2, 10, 11)]
```

```
estimate std.error statistic p.value conf.low conf.high df
```

```
1    -61.12    94.8   -0.6448  0.5371   -279.7    157.5   8
```

Then simulate with one randomization

Recall the true ATE:

```
trueATE <- with(sim_dat1, mean(y1 - y0))  
with(sim_dat1, mean(Y_Z_1 - Y_Z_0))
```

```
[1] 54
```

In one experiment (one simulation of the data) here are the estimates:

```
## Two ways to calculate the  
# difference of means estimator  
est_diff_means_1 <- with(sim_dat1, mean(Y[Z == 1]) - mean(Y[Z == 0]))  
est_diff_means_2 <- coef(lm_robust(Y ~ Z,  
  data = sim_dat1,  
  se = "classical"  
))["Z"]  
c(est_diff_means_1, est_diff_means_2)
```

```
[1] -123.6 -123.6
```

Then simulate with one randomization

In one experiment (one simulation of the data) here are the estimates:

```
## two ways to calculate the top-coded difference of means
sim_dat1$rankY <- rank(sim_dat1$Y)
sim_dat1$Y_tc <- with(sim_dat1, ifelse(rankY == max(rankY),
  Y[rankY == (max(rankY) - 1)], Y
))
est_topcoded_1 <- with(sim_dat1, mean(Y_tc[Z == 1]) - mean(Y_tc[Z == 0]))
est_topcoded_2 <- coef(lm_robust(Y_tc ~ Z,
  data = sim_dat1,
  se = "classical"
))["Z"]
c(est_topcoded_1, est_topcoded_2)
```

```
[1] -22.38 -22.37
```

Then simulate a different randomization and estimate the ATE with the same estimators

Now we can estimate with the same estimators using a **different** randomization. Notice that the answers differ. The estimators are estimating the *same estimand* but now they have a different randomization to work with.

```
# do another random assignment of the treatment in DeclareDesign
# this produces a new dataset with new assignments
sim_dat1 <- draw_data(base_design)
sim_dat1$rankY <- rank(sim_dat1$Y)
sim_dat1$Y_tc <- with(sim_dat1, ifelse(rankY == max(rankY),
  Y[rankY == (max(rankY) - 1)], Y
))
est_topcoded_1 <- with(sim_dat1, mean(Y_tc[Z == 1]) - mean(Y_tc[Z == 0]))
est_topcoded_2 <- coef(lm_robust(Y_tc ~ Z,
  data = sim_dat1,
  se = "classical"
))["Z"]
c(est_topcoded_1, est_topcoded_2)
```

```
[1] -16.75 -16.75
```

How do our estimators behave for this particular design?

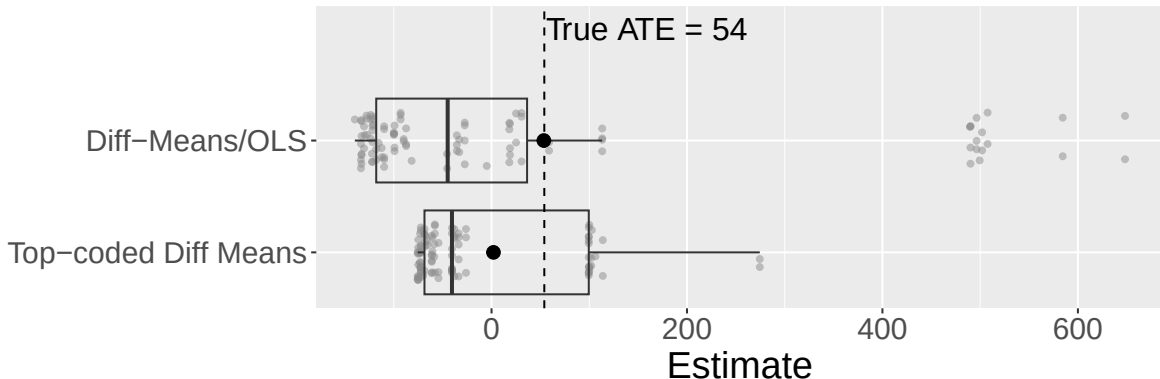
Our estimates vary across randomizations. Do our two estimators vary in the same ways?

```
## Combine into one DeclareDesign design object
## This has the base design, the estimand, and then our two estimators
design_plus_ests <- base_design + estimandATE + diff_means + diff_means_topcoded
## Run 10000 simulations (reassignments of treatment) and
## apply the two estimators (diff_means and diff_means_topcoded)
set.seed(12345)
diagnosis1 <- diagnose_design(design_plus_ests,
  bootstrap_sims = 0, sims = 10000
)
save(diagnosis1, file = "diagnosis1.rda")
```

	estimate	std.error	statistic	p.value	conf.low	conf.high	df	outcome
1	113.25	261.42	0.4332	0.67630	-489.6	716.1	8	Y
2	99.50	90.64	1.0977	0.30427	-109.5	308.5	8	newY
3	-127.37	258.71	-0.4924	0.63570	-724.0	469.2	8	Y
4	-66.12	94.38	-0.7006	0.50340	-283.8	151.5	8	newY
5	584.50	169.36	3.4512	0.00868	193.9	975.1	8	Y
6	-40.50	71.79	-0.5641	0.58812	-206.0	125.0	8	newY

How do our estimators behave for this particular design?

How should we interpret this plot (truth = dashed vertical line)?



Each gray point is the estimate from one possible experiment (100 of the 10,000).
The black dot marks the average of each estimator's estimates.

Which estimator is closer to the truth?

One criterion for choosing among estimators is to choose the one that is always **close to the truth**, regardless of the specific randomization.

An “unbiased” estimator is one for which **the average of the estimates across repeated designs** equals the truth (or $E_R(\hat{\tau}) = \bar{\tau}$). An unbiased estimator has “no systematic error” but also does not guarantee that we will be close to the truth.

One quantity that measures “closeness” to the truth is the **root mean squared error** (RMSE), which records the squared distances between the truth and the individual estimates.

Which estimator is closer to the truth?

Which estimator is better? (One is closer to the truth on average (RMSE) and is more precise. The other has no systematic error: it is unbiased.)

Estimator.1	Bias	RMSE	SD Estimate
Diff-Means/OLS	-0.47	245.46	245.47
Top-coded Diff Means	-52.00	101.68	87.38

There is no right answer.

Unbiased and biased estimators

Summary:

- We always have a *choice* of both estimands and estimators
- A good estimator should perform well regardless of the particular randomization under consideration from a given design. *Performs well* can mean “unbiased” and/or “low root mean squared error” (or “consistent”, which means that as the sample size increases the estimator gets closer to the truth).
- Simulation lets us learn how well an estimator works for a given study.

Section 4

Block randomization

Block-randomized experiments are a collection of mini-experiments

How should we define the **ATE** estimand in a block-randomized experiment?

If we think of the unit-level ATE as $(1/N) \sum_{i=1}^N Y_i(1) - Y_i(0)$, we could re-express this quantity equivalently using the ATE in block j , ATE_j , as follows:

$$ATE = \frac{1}{J} \sum_{j=1}^J \sum_{i=1}^{N_j} \frac{Y_i(1) - Y_i(0)}{N_j} = \sum_{j=1}^J \frac{N_j}{N} ATE_j$$

And it would be natural to *estimate* this quantity by plugging in what we can calculate:

$$\widehat{ATE} = \sum_{j=1}^J \frac{N_j}{N} \widehat{ATE}_j$$

Block-randomized experiments are a collection of mini-experiments

We can *define* the standard error of the estimator by also averaging the standard errors within each block (if our blocks are large enough)

$$SE(\widehat{ATE}) = \sqrt{\sum_{j=1}^J \left(\frac{N_j}{N}\right)^2 SE^2(\widehat{ATE}_j)}$$

Estimating the ATE in block-randomized experiments

One estimation approach simply replaces ATE_j with $\widehat{ATE}$:

```
## Shrink the extreme observation so that the plots below look better  
dat$y0[dat$y0 == 1000] <- 300  
dat$y1 <- with(dat, y0 + tau)  
dat$Y <- with(dat, Z * y1 + (1 - Z) * y0)  
with(dat, table(b, Z))
```

```
      Z  
b    0 1  
a  4 2  
b  2 2
```

As we can see, we have 6 units in block a, 2 of which are assigned to treatment, and 4 units in block b, 2 of which are assigned to treatment.

The ATE in block-randomized experiments

One estimation approach simply replaces ATE_j with $\widehat{ATE}$. First see how we **calculate** the true ATE :

```
datb <- dat %>%  
  group_by(b) %>%  
  summarize(  
    nb = n(), pb = mean(Z), estateb = mean(Y[Z == 1]) - mean(Y[Z == 0]),  
    ateb = mean(y1 - y0), .groups = "drop"  
  )  
datb
```

```
# A tibble: 2 x 5  
  b      nb    pb estateb ateb  
  <chr> <int> <dbl>   <dbl> <dbl>  
1 a         6 0.333    16.8     60  
2 b         4 0.5     296.     45
```

```
## The true ATE  
with(dat, mean(y1 - y0))
```

```
[1] 54
```

```
## Another way to calculate the true ATE  
with(datb, sum(ateb * (nb / sum(nb))))
```

```
[1] 54
```

Estimating the ATE in block-randomized experiments

And we can estimate the overall ATE using block-size weights:

```
## Show that difference_in_means uses the block-size weights.  
e1 <- with(datb, sum(estateb * (nb / sum(nb))))  
e1
```

```
[1] 128.2
```

```
dat <- dat %>%  
  group_by(b) %>%  
  mutate(  
    pb = mean(Z),  
    nbwt = Z / (pb) + (1 - Z) / (1 - pb)  
  )  
  
e2 <- lm_robust(Y ~ Z, data = dat, weights = nbwt)  
coef(e2)[["Z"]]
```

```
[1] 128.3
```

Estimating the ATE in block-randomized experiments

```
## A more general (but newer) R package: https://benbhansen-stats.github.io/propertee/index.html  
## Install once from GitHub; skip if already installed so the slides render offline  
if (!requireNamespace("propertee", quietly = TRUE)) {  
  remotes::install_github("benbhansen-stats/propertee", dependencies = TRUE)  
}  
library(propertee)  
blocked_design <- rct_spec(Z ~ unit_of_assignment(id) + block(b), data = dat)  
e2a <- lm_robust(Y ~ Z, data = dat, weights = ate(blocked_design, data = dat))  
coef(e2a)[["Z"]]
```

```
[1] 128.3
```

```
e2b <- lmitt(Y ~ 1, specification = blocked_design, data = dat, weights = "ate")  
coef(e2b)[["Z."]]
```

```
[1] 128.3
```

Estimating the ATE in block-randomized experiments

Notice that this is **not** the same as the following:

```
## Ignoring the blocks
e3 <- lm(Y ~ Z, data = dat)
coef(e3)[["Z"]]
```

```
[1] 156.8
```

```
## Then using "precision weights"
e4 <- lm(Y ~ Z + block, data = dat)
coef(e4)[["Z"]]
```

```
[1] 136.2
```

```
e4a <- lm_robust(Y ~ Z, fixed_effects = ~block, data = dat)
coef(e4a)[["Z"]]
```

```
[1] 136.2
```

```
## Or with the absorb=TRUE argument:
e4b <- lmitt(Y ~ 1, specification = blocked_design, data = dat, absorb = TRUE)
coef(e4b)[["Z."]]
```

```
[1] 136.2
```

How do they differ? (The first ignores the blocks. The second uses a different set of weights, created by use of “fixed effects” or “indicator” or “dummy” variables.)

Which estimator should we use?

Each of the three estimators produces a different estimate (imagining that they all target the same estimand):

```
c(coef(e2)[["Z"]], coef(e3)[["Z"]], coef(e4)[["Z"]])
```

```
[1] 128.3 156.8 136.2
```

Which estimator should we use?

Each of the three estimators produces a different estimate (imagining that they all target the same estimand):

Which estimator should we use for this design? We can answer this question by simulation in DeclareDesign.

```
## declare a new base design that includes the block indicator b
base_design_blocks <-
  # declare the population
  declare_population(dat[, c("b", "y0", "y1", "id")]) +
  # Tell DD that b indicates blocks and
  # 2 units are treated in each block
  declare_assignment(
    Z = conduct_ra(N = N, m = 2, blocks = b),
    Z_cond_prob = obtain_condition_probabilities(assignment = Z, m = 2)
  ) +
  # relationship between potential outcomes and observed outcomes
  declare_potential_outcomes(Y ~ Z * y1 + (1 - Z) * y0) +
  # observed outcome and treatment assignment
  declare_reveal(Y, Z)
```

Which estimator should we use?

```
# the estimand is the average treatment effect
estimandATEb <- declare_inquiry(ATE = mean(Y_Z_1 - Y_Z_0))

# two different estimators (the others are defined below)
est1 <- declare_estimator(Y ~ Z,
  inquiry = estimandATEb, .method = lm_robust,
  label = "Ignores Blocks"
)
est2 <- declare_estimator(Y ~ Z,
  inquiry = estimandATEb, .method = difference_in_means, blocks = b,
  label = "DiM: Block-Size Weights"
)
```

Which estimator should we use?

```
nbwt_est_fun <- function(data) {  
  data <- data %>%  
    group_by(b) %>%  
    mutate(  
      pb = mean(Z),  
      newnbwt = (Z / pb) + ((1 - Z) / (1 - pb))  
    )  
  obj <- lm_robust(Y ~ Z, data = data, weights = newnbwt)  
  res <- tidy(obj) %>% filter(term == "Z")  
  return(res)  
}  
  
est3 <- declare_estimator(  
  handler = label_estimator(nbwt_est_fun), inquiry = estimandATEb,  
  label = "LM: Block Size Weights"  
)
```


Which estimator should we use?

```
propertee_est_fun <- function(data) {  
  des <- rct_spec(Z ~ unit_of_assignment(id) + block(b), data = data)  
  obj <- lm_robust(Y ~ Z, data = data, weights = ate(des, data = data))  
  res <- tidy(obj) %>% filter(term == "Z")  
  return(res)  
}  
  
est3a <- declare_estimator(  
  handler = label_estimator(propertee_est_fun), inquiry = estimandATEb,  
  label = "Propertee: Block Size Weights"  
)
```

Which estimator should we use?

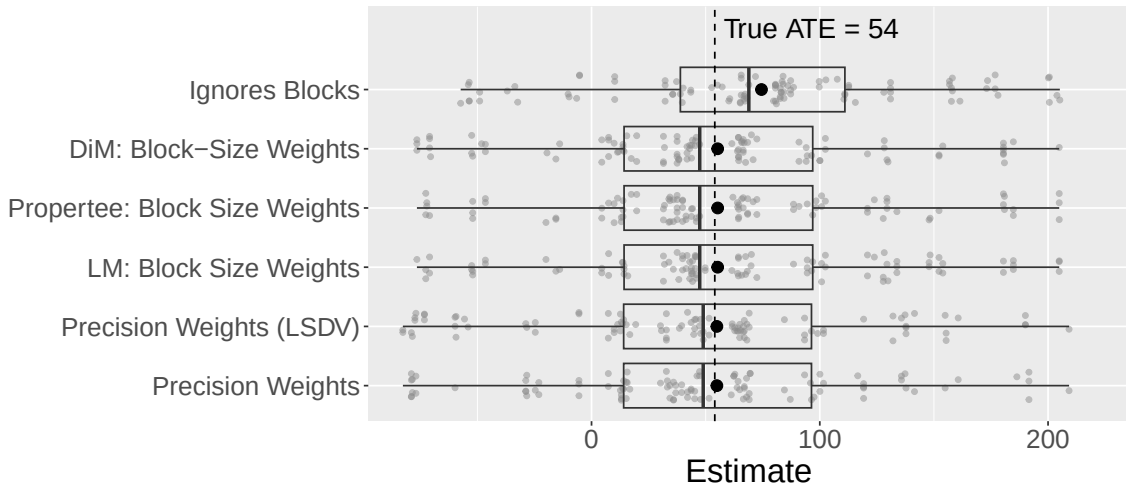
```
# two more estimators
est4 <- declare_estimator(Y ~ Z,
  inquiry = estimandATEb,
  .method = lm_robust, fixed_effects = ~b, label = "Precision Weights"
)
est5 <- declare_estimator(Y ~ Z + b,
  inquiry = estimandATEb,
  .method = lm_robust, label = "Precision Weights (LSDV)"
)
```

```
# The new design object has
# the base design, the estimand, and six estimators
design_blocks <- base_design_blocks + estimandATEb +
  est1 + est2 + est3 + est3a + est4 + est5
```

Then we will run 10,000 simulations (reassign treatment 10,000 times) and summarize the estimates produced by each of these six estimators.

Which estimator should we use?

How should we interpret this plot?



Each gray point is the estimate from one possible experiment (100 of the 10,000); the black dot marks the average.

Which estimator is closer to the truth?

Which estimator works better for these particular data and this design?

Estimator.1	Bias	RMSE	SD Estimate	Coverage
DiM: Block-Size Weights	1.26	64.86	64.85	0.75
Ignores Blocks	20.45	68.99	65.89	0.98
LM: Block Size Weights	1.26	64.86	64.85	0.99
Precision Weights	0.92	68.35	68.35	0.93
Precision Weights (LSDV)	0.92	68.35	68.35	0.93
Propertee: Block Size Weights	1.26	64.86	64.85	0.99

What is the problem with coverage?

Notice that the estimated standard error from `difference_in_means` is too low:

```
# A tibble: 6 x 3
```

	estimator	mean_est_se	sd_est
	<chr>	<dbl>	<dbl>
1	DiM: Block-Size Weights	61.8	64.8
2	Ignores Blocks	74.5	65.9
3	LM: Block Size Weights	73.9	64.8
4	Precision Weights	69.5	68.3
5	Precision Weights (LSDV)	69.5	68.3
6	Propertee: Block Size Weights	73.9	64.8

When is the “precision weights” bias large?

- The fixed-effects / LSDV estimator weights each block by $n_j p_j (1 - p_j)$ — the “precision weights” — instead of weighting by size n_j . If the treated share p_j is the same in every block, the two sets of weights coincide and there is no bias.
- The bias appears when *two* conditions hold at once: p_j varies across blocks *and* the block-level ATE_j varies along with p_j .
- In our example the bias was tiny (0.92 against an ATE of 54): the block ATEs are similar (60 and 45) and the two sets of weights nearly equal (.57/.43 versus .60/.40).
- Let's build two designs: one where both conditions hold strongly (design A) and one where the effect is the same in every block (design B).

Two designs to see the bias (and the precision gain) I

```
## Two blocks of 40 units. In B1 we treat half (20 of 40);  
## in B2 we treat very few (2 of 40): p_1 = .5, p_2 = .05.  
## The precision weights come out .84/.16; the size weights, .5/.5.  
set.seed(20260729)  
datAB <- data.frame(  
  b = rep(c("B1", "B2"), each = 40),  
  y0 = rnorm(80, mean = 0, sd = 5),  
  id = 1:80  
)  
## Design A: the block ATE varies ALONG WITH p_j (10 in B1, 0 in B2)  
datAB$tau_A <- ifelse(datAB$b == "B1", 10, 0)  
## Design B: the same effect in both blocks  
datAB$tau_B <- 10  
assign_AB <- declare_assignment(  
  Z = conduct_ra(N = 80, blocks = b, block_m = c(20, 2))  
)
```

Two designs to see the bias (and the precision gain) II

```
est_dim <- declare_estimator(Y ~ Z,
  .method = difference_in_means, blocks = b,
  inquiry = "ATE", label = "DiM: Block-Size Weights"
)
est_fe <- declare_estimator(Y ~ Z,
  .method = lm_robust, fixed_effects = ~b,
  inquiry = "ATE", label = "Precision Weights (FE)"
)
design_A <- declare_population(datAB) + assign_AB +
  declare_potential_outcomes(Y ~ Z * (y0 + tau_A) + (1 - Z) * y0) +
  declare_reveal(Y, Z) + declare_inquiry(ATE = mean(tau_A)) +
  est_dim + est_fe
design_B <- declare_population(datAB) + assign_AB +
  declare_potential_outcomes(Y ~ Z * (y0 + tau_B) + (1 - Z) * y0) +
  declare_reveal(Y, Z) + declare_inquiry(ATE = mean(tau_B)) +
  est_dim + est_fe
```


Two designs to see the bias (and the precision gain) III

10,000 simulations of each design:

Design	Estimator	Mean Estimand	Bias	RMSE	SD Estimate
design_A	DiM: Block-Size Weights	5.00	0.03	2.08	2.08
design_A	Precision Weights (FE)	5.00	3.42	3.73	1.50
design_B	DiM: Block-Size Weights	10.00	-0.00	2.09	2.09
design_B	Precision Weights (FE)	10.00	-0.01	1.49	1.50

Lessons from the two designs

- In design A the precision weights produce a bias of about 3.4 against a true ATE of 5 — a systematic error of nearly 70% that does *not* go away as the sample grows. The block-size weights remain unbiased.
- In design B — the same design, but with the effect equal in both blocks — both estimators are unbiased and the precision weights deliver on their promise: a standard deviation of about 1.5 versus 2.1, because they down-weight the block where only 2 units were treated.
- The lesson: when p_j differs across blocks, block-size weights are the safe choice for estimating the ATE; precision weights buy lower variance at the price of a bias that depends on how much ATE_j varies with p_j — something we cannot check in the observed data.

Section 5

Cluster randomization

In cluster-randomized experiments, units are randomized as a group (cluster) to treatment I

- **Example 1:** an intervention is randomized across neighborhoods, so **all** households in a neighborhood will be assigned to the same treatment condition, but different neighborhoods will be assigned different treatment conditions.
- **Example 2:** an intervention is randomized across people and each person is measured four times after treatment, so our data contain four rows per person.
- **Not An Example 1:** Neighborhoods are chosen for the study. Within each neighborhood about half of the people are assigned to treatment and half to control. (What kind of study is this? It is not a cluster-randomized study.)
- **Not an Example 2:** an intervention is randomized to some neighborhoods and not to others, the outcomes include measurements of neighborhood-level trust in government and total land area in the neighborhood devoted to gardens. (Sometimes a cluster randomized experiment can be turned into a simple randomized experiment. Or may contain more than one possible approach to analysis and interpretation.)

In cluster-randomized experiments, units are randomized as a group (cluster) to treatment II

How might the distribution of test statistics and estimators differ between an experiment where individual units are randomized and one where clusters are randomized?

Estimating the ATE in cluster-randomized experiments

Bias problems in cluster-randomized experiments:

- When clusters are the same size, the usual difference-in-means estimator is unbiased.
- But be careful when clusters have different numbers of units or you have very few clusters because then treatment effects may be correlated with cluster size.
- When cluster size is related to potential outcomes, the usual difference-in-means estimator is biased.
<https://declaredesign.org/blog/bias-cluster-randomized-trials.html>

Estimating the SE for the ATE in cluster-randomized experiments I

- **Misleading statistical inferences:** In general, the default standard error understates the uncertainty in such designs and thus produces tests with false positive rates that are too high (or, equivalently, confidence intervals with coverage rates that are too low).
- The “cluster robust standard errors” implemented in common software work well **when the number of clusters is large** (more than 50 in some cases).
- The default cluster standard errors in `lm_robust` (the CR2 SEs) work better than those available in Stata (as of this writing).
- The wild bootstrap helps control error rates but gives up statistical power much more than perhaps necessary in a cluster-randomized study where direct randomization inference is possible.
- When in doubt, one can calculate p -values by direct simulation (direct randomization-based inference) to assess whether the cluster robust estimates are correct.

Overall, if you have any worries or doubts, simulate to study the performance of your estimators, tests, and confidence intervals.

An example of estimation

Imagine we had data from 10 clusters with either 100 people (in 2 clusters) or 10 people per cluster (in 8 clusters). The total size of the data is 280.

```
# A tibble: 6 x 6
# Groups:   clus_id [2]
  clus_id indiv Y_Z_0 Y_Z_1     Z     Y
  <chr>   <chr> <dbl> <dbl> <int> <dbl>
1 01      010   4.51  4.61     0  4.51
2 01      035   4.63  4.73     0  4.63
3 01      068   4.76  4.86     0  4.76
4 03      205   3.13  4.13     1  4.13
5 03      206   2.41  3.41     1  3.41
6 03      208   2.95  3.95     1  3.95
```


An example of estimation

Which estimator should we use? Which test should we use? On what basis should we choose among the alternatives?

```
lmc1 <- lm_robust(Y ~ Z, data = dat1)
lmc2 <- lm_robust(Y ~ Z, clusters = clus_id, data = dat1)
lmc3 <- lm_robust(Y ~ Z + cl_sizeF, clusters = clus_id, data = dat1)
tidy(lmc1)[2, ]
```

	term	estimate	std.error	statistic	p.value	conf.low	conf.high	df	outcome
2	Z	0.3024	0.1207	2.504	0.01284	0.06471	0.5401	278	Y

```
tidy(lmc2)[2, ]
```

	term	estimate	std.error	statistic	p.value	conf.low	conf.high	df	outcome
2	Z	0.3024	1.079	0.2804	0.796	-2.969	3.574	3.282	Y

```
tidy(lmc3)[2, ]
```

	term	estimate	std.error	statistic	p.value	conf.low	conf.high	df	outcome
2	Z	0.3024	0.306	0.9882	0.4386	-1.194	1.799	1.769	Y

Use simulation to assess estimators and tests

If you look at the code for the slides, you will see that we simulate the design many times, each time calculating an estimate and a confidence interval for different estimators of the ATE.

What can we learn from this table? (Coverage? `sd_estimate` versus `mean_se`?)

Table 1: Estimator and test performance in 10000 simulations of the cluster randomized design for different estimators and confidence intervals

estimator	coverage	sd_estimate	mean_se
$Y \sim Z * I(\text{clus_size} - \text{mean}(\text{clus_size})), \text{CR2}$	0.75	1.60	0.06
$Y \sim Z, \text{cl_size fe}, \text{CR2}$	0.75	0.36	0.30
$Y \sim Z, \text{CR2}$	0.56	1.09	0.72
$Y \sim Z, \text{HC2}$	0.56	1.09	0.13
$Y \sim Z, \text{IID}$	0.56	1.09	0.12
$Y \sim Z * I(\text{cl_size} - \text{mean}(\text{cl_size})), \text{CR2}$	0.75	0.36	0.30
$Y \sim Z + \text{cl_sizeF}, \text{CR2}$	0.56	1.24	0.83

Use simulation to assess estimators and tests

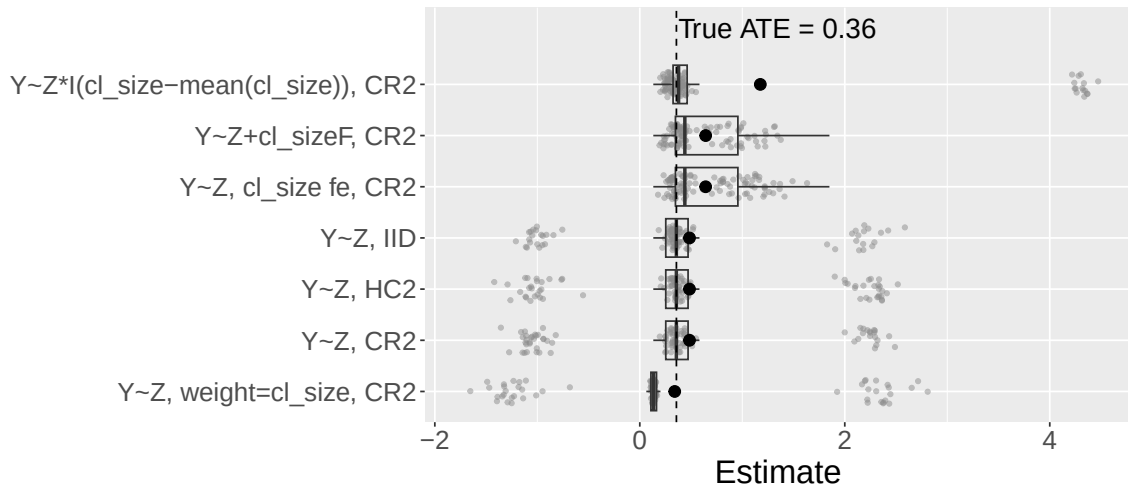
What can we learn from this table? (Bias? Closeness to the truth?)

Table 2: Estimator and test performance in 10000 simulations of the cluster randomized design for different estimators and confidence intervals

estimator	bias	rmse
$Y \sim Z * I(\text{clus_size} - \text{mean}(\text{clus_size})), \text{CR2}$	0.819	1.799
$Y \sim Z, \text{cl_size fe}, \text{CR2}$	0.285	0.458
$Y \sim Z, \text{CR2}$	0.128	1.097
$Y \sim Z, \text{HC2}$	0.128	1.097
$Y \sim Z, \text{IID}$	0.128	1.097
$Y \sim Z * I(\text{cl_size} - \text{mean}(\text{cl_size})), \text{CR2}$	0.285	0.458
$Y \sim Z + \text{cl_sizeF}, \text{CR2}$	-0.016	1.243

Use simulation to assess estimators and tests

How should we interpret this plot?



Each gray point is the estimate from one possible experiment (100 of the 10,000); the black dot marks the average.

Summary of estimation and testing in cluster-randomized trials

- Cluster randomized trials pose a series of special problems for standard approaches to estimation and testing.
- If randomization occurs at the cluster level, then the uncertainty arising from randomization is also at the cluster level.
- When we have enough clusters, the “cluster robust” standard errors can help us produce confidence intervals with correct coverage. **Cluster robust standard errors require many clusters.**
- If cluster size (or some characteristic) is related to effect size, then estimation can be biased (and we need to adjust somehow).

Section 6

Binary outcomes

Binary outcomes: Set up our data for simulation in DeclareDesign

```
# population size
N <- 100
# declare the population
thepop_bin <- declare_population(
  N = N, x1 = draw_binary(prob = .5, N = N),
  x2 = rnorm(N)
)
# declare the potential outcomes on the log-odds scale: x1 STRONGLY
# predicts the outcome (coefficient 3). That strong covariate is what
# separates the conditional odds ratio from the marginal one
# (non-collapsibility), the bias Freedman (2008) warns about below.
thepo_bin <- declare_potential_outcomes(Y ~ rbinom(
  n = N, size = 1,
  prob = plogis(-1.5 + 1 * Z + 3 * x1)
))
# two possible target quantities:
# difference in means or difference in log-odds
thetarget_ate <- declare_inquiry(ate = mean(Y_Z_1 - Y_Z_0))
thetarget_logodds <- declare_inquiry(
  logodds = log(mean(Y_Z_1) / (1 - mean(Y_Z_1))) -
    log(mean(Y_Z_0) / (1 - mean(Y_Z_0)))
)
```

Binary outcomes: Set up our data for simulation in DeclareDesign

```
# declare how treatment is assigned
# m units are assigned to levels of treatment Z
theassign_bin <- declare_assignment(Z = conduct_ra(N = N, m = floor(N / 3)))
# declare which outcome values are revealed for different values of Z
thereveal_bin <- declare_reveal(Y, Z)
# pull it all together: population, potential outcomes, assignment,
## outcome Y connected to Z
des_bin <- thepop_bin + thepo_bin + theassign_bin + thereveal_bin
# one realization of the data (randomize treatment once)
set.seed(12345)
dat2 <- draw_data(des_bin)
```


Binary outcomes: Estimands I

How should we interpret the following true quantities or estimands? (Y_{Z_1} , Y_{Z_0} are potential outcomes, Y is observed, x_1 , x_2 are covariates, Z is treatment assignment. Here $N=100$.)

```
## Look at the first 6 observations  
head(dat2[, -7])
```

	ID	x1	x2	Y_{Z_0}	Y_{Z_1}	Z
1	001	1	-0.54039	1	1	0
2	002	1	1.94769	1	1	0
3	003	1	0.05359	1	1	0
4	004	1	0.35166	1	1	1
5	005	0	-0.67098	0	1	0
6	006	0	0.27795	0	0	1

Binary outcomes: Estimands II

How should we interpret the following true quantities or estimands? (Y_{Z_1} , Y_{Z_0} are potential outcomes, Y is observed, x_1 , x_2 are covariates, Z is treatment assignment. Here $N=100$.)

```
ate_bin <- with(dat2, mean(Y_Z_1 - Y_Z_0))
bary1 <- mean(dat2$Y_Z_1)
bary0 <- mean(dat2$Y_Z_0)
diff_log_odds_bin <- with(
  dat2,
  log(bary1 / (1 - bary1)) - log(bary0 / (1 - bary0))
)
c(
  bary1 = bary1, bary0 = bary0, true_ate = ate_bin,
  true_diff_log_odds = diff_log_odds_bin
)
```

bary1	bary0	true_ate	true_diff_log_odds
0.6800	0.5300	0.1500	0.6336

Binary outcomes: Estimands III

Do you want to estimate the difference in log-odds?

$$\delta = \log \frac{\bar{y}_1}{1 - \bar{y}_1} - \log \frac{\bar{y}_0}{1 - \bar{y}_0} \quad (1)$$

Or the difference in proportions?

$$\bar{\tau} = \bar{y}_1 - \bar{y}_0 \quad (2)$$

Recall that $\bar{y}_1$ is the *proportion* of $y_1 = 1$ in the data.

Freedman (2008b) shows us that the covariate-adjusted logit coefficient estimator is a biased estimator of the difference in log-odds estimand. He also shows a consistent estimator of that same estimand.

We know that the difference of proportions in the sample should be an unbiased estimator of the difference of proportions.

An example of estimation I

How should we interpret the following estimates? (What does the difference of means estimator require in terms of assumptions? What does the logistic regression estimator require in terms of assumptions?)

```
lmbin1 <- lm_robust(Y ~ Z, data = dat2)
glmbin1 <- glm(Y ~ Z, data = dat2, family = binomial(link = "logit"))

broom::tidy(lmbin1)[2, ]
```

	term	estimate	std.error	statistic	p.value	conf.low	conf.high	df	outcome
2	Z	-0.05156	0.1067	-0.4831	0.6301	-0.2634	0.1603	98	Y

```
broom::tidy(glmbin1)[2, ]
```

A tibble: 1 x 5

	term	estimate	std.error	statistic	p.value
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	Z	-0.211	0.429	-0.491	0.623

An example of estimation II

What about covariates? Why do we use covariates in randomized experiments?

```
lmbin2 <- lm_robust(Y ~ Z + x1, data = dat2)
glmbin2 <- glm(Y ~ Z + x1, data = dat2, family = binomial(link = "logit"))

tidy(lmbin2)[2, ]
```

```
      term estimate std.error statistic p.value conf.low conf.high df outcome
2      Z  0.04569   0.07911    0.5776  0.5649  -0.1113   0.2027  97      Y

tidy(glmbin2)[2, ]
```

```
# A tibble: 1 x 5
  term estimate std.error statistic p.value
<chr>    <dbl>    <dbl>    <dbl>    <dbl>
1 Z      0.344    0.593    0.580    0.562
```

An example of estimation III

Now let's compare our estimates

```
c(  
  dim = coef(lmbin1)[["Z"]],  
  dim_x1 = coef(lmbin2)[["Z"]],  
  glm = coef(glmbin1)[["Z"]],  
  glm_x1 = coef(glmbin2)[["Z"]]  
)
```

dim	dim_x1	glm	glm_x1
-0.05156	0.04569	-0.21072	0.34357

Unadjusted versus adjusted logit: different targets

- The logit with only Z is *saturated*: its coefficient is exactly the observed difference in log-odds, $\text{logit}(\bar{Y}_{Z=1}) - \text{logit}(\bar{Y}_{Z=0})$. It is a consistent estimator of the marginal difference δ .
- Adding a covariate changes the target: the coefficient on Z in $\text{glm}(Y \sim Z + x1)$ aims at the difference in log-odds *holding $x1$ fixed* — a conditional quantity.
- Odds ratios are not *collapsible*: the conditional difference in log-odds sits farther from zero than the marginal one, even with no confounding. So the adjusted coefficient is a biased estimator of δ , and that gap does not shrink as N grows (Freedman 2008b).
- Freedman's repair (the “plug-in”): with the *same* adjusted model, predict each unit's probability under $Z = 1$ and under $Z = 0$, average, and take the difference in log-odds. That estimator returns to the marginal scale and is consistent.

An example of estimation: The Freedman estimators I

No covariate:

```
freedman_plugin_estfn1 <- function(data) {  
  glmbin <- glm(Y ~ Z, data = data, family = binomial(link = "logit"))  
  preddat <- data.frame(Z = rep(c(0, 1), nrow(data)))  
  preddat$yhat <- predict(glmbin, newdata = preddat, type = "response")  
  bary1 <- mean(preddat$yhat[preddat$Z == 1])  
  bary0 <- mean(preddat$yhat[preddat$Z == 0])  
  diff_log_odds <- log(bary1 / (1 - bary1)) - log(bary0 / (1 - bary0))  
  return(data.frame(estimate = diff_log_odds, estimator = "Freedman:Z", inquiry = "logodds"))  
}
```

```
freedman_plugin_lm_estfn1 <- function(data) {  
  lmbin <- lm(Y ~ Z, data = data)  
  preddat <- data.frame(Z = rep(c(0, 1), nrow(data)))  
  preddat$yhat <- predict(lmbin, newdata = preddat)  
  bary1 <- mean(preddat$yhat[preddat$Z == 1])  
  bary0 <- mean(preddat$yhat[preddat$Z == 0])  
  diff_log_odds <- log(bary1 / (1 - bary1)) - log(bary0 / (1 - bary0))  
  return(data.frame(estimate = diff_log_odds, estimator = "Freedman,LM:Z", inquiry = "logodds"))  
}
```


An example of estimation: The Freedman estimators II

With covariate:

```
freedman_plugin_estfn2 <- function(data) {  
  N <- nrow(data)  
  glmbin <- glm(Y ~ Z + x1, data = data, family = binomial(link = "logit"))  
  preddat <- data.frame(Z = rep(c(0, 1), each = N))  
  preddat$x1 <- rep(data$x1, 2)  
  preddat$yhat <- predict(glmbin, newdata = preddat, type = "response")  
  bary1 <- mean(preddat$yhat[preddat$Z == 1])  
  bary0 <- mean(preddat$yhat[preddat$Z == 0])  
  diff_log_odds <- log(bary1 / (1 - bary1)) - log(bary0 / (1 - bary0))  
  return(data.frame(estimate = diff_log_odds, estimator = "Freedman:Z,x1", inquiry = "logodds"))  
}
```

```
freedman_plugin_lm_estfn2 <- function(data) {  
  N <- nrow(data)  
  lmbin <- lm(Y ~ Z + x1, data = data)  
  preddat <- data.frame(Z = rep(c(0, 1), each = N))  
  preddat$x1 <- rep(data$x1, 2)  
  preddat$yhat <- predict(lmbin, newdata = preddat)  
  bary1 <- mean(preddat$yhat[preddat$Z == 1])  
  bary0 <- mean(preddat$yhat[preddat$Z == 0])  
  diff_log_odds <- log(bary1 / (1 - bary1)) - log(bary0 / (1 - bary0))  
  return(data.frame(estimate = diff_log_odds, estimator = "Freedman,LM:Z,x1", inquiry = "logodds"))  
}
```

Now let's compare the estimates from the eight different estimators

tmle: an easy way to implement the Freedman plug-in

- The `tmle` package implements targeted maximum likelihood estimation. Its first step is exactly the Freedman plug-in: fit a logistic model (`Qform`), predict each unit under $Z = 1$ and under $Z = 0$, and average.
- Its second step (“targeting”) corrects the initial fit using the treatment probability. In a randomized experiment that probability is constant (`gform = A ~ 1`), and with a logistic model containing an intercept and a treatment term fit by maximum likelihood the correction is exactly zero: the targeting step’s “clever covariate” is already a linear combination of the logistic score functions.
- The result: `tmle()` reports the *same* number as the Freedman plug-in — and it adds standard errors and confidence intervals based on the influence curve, which our hand-written functions do not provide.

tmle = Freedman plug-in: numerical check

The two estimators of the same log-odds estimand, using the same fitted model ($Y \sim Z + x1$):

```
c(  
  freedman_plugin = freedman_plugin_estfn2(dat2)[["estimate"]],  
  tmle_log_odds = tmle2$estimates$OR$log.psi  
)
```

```
freedman_plugin  tmle_log_odds  
      0.1818      0.1818
```

```
## tmle adds the confidence interval for the odds ratio:  
tmle2$estimates$OR$CI
```

```
[1] 0.6328 2.2731
```

```
## and it also reports the difference in proportions (ATE) with its interval:  
c(tmle_ate = tmle2$estimates$ATE$psi)
```

```
tmle_ate  
  0.04405  
tmle2$estimates$ATE$CI
```

```
[1] -0.1097  0.1978
```

An example of using DeclareDesign to assess our estimators I

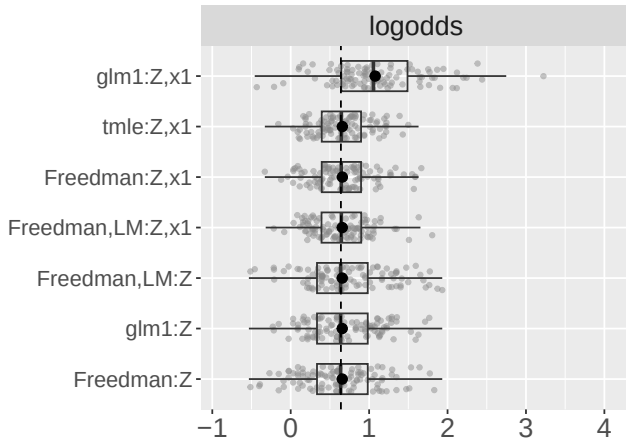
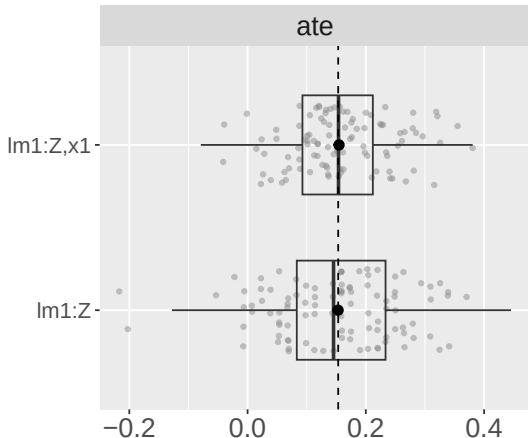
```
# declare 4 estimators for DD
# first estimator: linear regression with ATE as the target quantity
estb1 <- declare_estimator(Y ~ Z,
  .method = lm_robust, label = "lm1:Z",
  inquiry = thetarget_ate
)
# second estimator: linear regression with a covariate, with ATE as the target quantity
estb2 <- declare_estimator(Y ~ Z + x1,
  .method = lm_robust, label = "lm1:Z,x1",
  inquiry = thetarget_ate
)
# third estimator: logistic regression with log-odds as the target quantity
estb3 <- declare_estimator(Y ~ Z,
  .method = glm, family = binomial(link = "logit"),
  label = "glm1:Z", inquiry = thetarget_logodds
)
# fourth estimator: logistic regression with a covariate, with log-odds as the target quantity
estb4 <- declare_estimator(Y ~ Z + x1,
  .method = glm, family = binomial(link = "logit"),
  label = "glm1:Z,x1", inquiry = thetarget_logodds
)
```

An example of using DeclareDesign to assess our estimators II

```
# Pull it all together: des_bin is
# the population, the potential outcomes,
# the assignment, the realized values of Y connected to Z.
# We also add two target quantities and nine estimators.
des_bin_plus_est <- des_bin + thetarget_ate + thetarget_logodds +
  estb1 + estb2 + estb3 + estb4 + custom_tmle_estimator +
  freedman_est1 + freedman_est2 + freedman_est3 + freedman_est4
```

Using simulation to assess our estimators

How should we interpret this plot? (Differences in scales make it difficult.)



Estimate

The dashed line marks the true value of each estimand.

The gray points are 100 of the 1,000 estimates; the black dot marks the average.

Which estimator is closer to the truth?

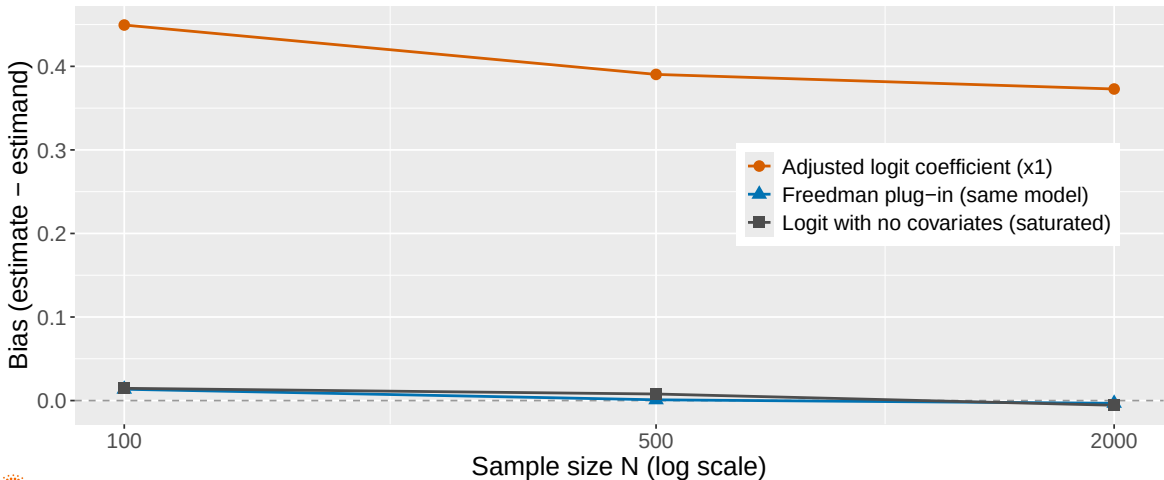
Which estimator works better for this design and these data?

Table 3: Estimator and test performance in 1000 simulations of the randomized design for different estimators.

est	estimand	bias	rmse
Freedman,LM:Z	logodds	0.017	0.405
Freedman,LM:Z,x1	logodds	0.018	0.292
Freedman:Z	logodds	0.017	0.405
Freedman:Z,x1	logodds	0.018	0.293
glm1:Z	logodds	0.017	0.405
glm1:Z,x1	logodds	0.437	0.674
lm1:Z	ate	0.000	0.090
lm1:Z,x1	ate	0.001	0.065
tmle:Z,x1	logodds	0.018	0.293

The adjusted logit coefficient's bias does not shrink with N

The same design as above with 500 simulations per N: the adjusted coefficient's bias stays put while the other two estimators converge on the estimand.



Section 7

Covariance adjustment

Covariates are for precision in experiments — but “controlling for” is not free

- Randomization balances covariates on average: we do not need x to identify the ATE. A covariate that predicts the outcome can still shrink our standard errors — that is why we adjust.
- The reflex from observational studies — report the coefficient on Z in $\text{lm}(Y \sim Z + x)$ — is a different estimator, and Freedman (2008a) showed it is a **biased** estimator of the ATE. The bias shrinks as N grows (Lin 2013), so the question is when it matters and what to do about it.
- Plan: build a small experiment where the bias is visible, compare four estimators, then watch the bias fade as the experiment grows.

A small experiment with a skewed, prognostic covariate

```
set.seed(5)
N3 <- 20 # the size of a pilot study or of a cluster-level analysis
## A skewed covariate: most units small, a few far out in the tail
x <- exp(rnorm(N3, mean = 0, sd = 1))
## x strongly predicts the outcome under control, so
## adjustment promises a real precision gain
y0 <- x + rnorm(N3, sd = 0.5)
## Treatment effects are biggest exactly where x is extreme. Freedman's
## bias grows with this covariance between leverage and effect size
y1 <- y0 + 0.75 * x^2
dat3 <- data.frame(x = x, y0 = y0, y1 = y1)
trueATE3 <- mean(dat3$y1 - dat3$y0)
c(ATE = trueATE3, cor_x_y0 = cor(dat3$x, dat3$y0), max_x = max(dat3$x))
```

ATE	cor_x_y0	max_x
2.5112	0.9275	5.5369

10 of the 20 units will be assigned to treatment:

```
base_design_cov <- declare_population(dat3) +
  declare_assignment(Z = conduct_ra(N = N, m = 10)) +
  declare_potential_outcomes(Y ~ Z * y1 + (1 - Z) * y0) +
  declare_reveal(Y, Z)
```

Three familiar estimators of the same ATE

```
estimandATEc <- declare_inquiry(ATE = mean(Y_Z_1 - Y_Z_0))
est_c1 <- declare_estimator(Y ~ Z,
  inquiry = estimandATEc, .method = difference_in_means,
  label = "Diff means: ignores x"
)
est_c2 <- declare_estimator(Y ~ Z + x,
  inquiry = estimandATEc, .method = lm_robust,
  label = "Y~Z+x: 'controls for' x"
)
## Lin's repair of Freedman's efficiency critique: center x, interact
## with Z, use robust standard errors
est_c3 <- declare_estimator(Y ~ Z,
  covariates = ~x, inquiry = estimandATEc, .method = lm_lin,
  label = "lm_lin: Y~Z*(x-mean(x))"
)
```

lm_lin is the plug-in principle applied to the linear model

The same move as the Freedman plug-in for logit above: fit the adjusted model, predict each unit's outcome under $Z = 1$ and under $Z = 0$, average, take the difference. For a linear model with interactions, the plug-in **is** `lm_lin` — two names, one estimator.

```
set.seed(20260731)
dat3sim <- draw_data(base_design_cov)
lin_fit <- lm_lin(Y ~ Z, covariates = ~x, data = dat3sim)
ifit <- lm(Y ~ Z * x, data = dat3sim)
p1 <- predict(ifit, newdata = transform(dat3sim, Z = 1))
p0 <- predict(ifit, newdata = transform(dat3sim, Z = 0))
c(lm_lin = coef(lin_fit)[["Z"]], plugin = mean(p1) - mean(p0))
```

```
lm_lin plugin
1.567 1.567
```

(Without the interaction the plug-in from `lm(Y ~ Z + x)` just returns the coefficient on Z , so the naive estimator gains nothing from this move.)

The remaining bias can be estimated from observed data — then subtracted

Chang et al. (2024) write out the exact bias of the Lin estimator — covariances of each unit's *leverage* with its outcome — and every term is estimable without bias, so subtracting the estimate yields an exactly unbiased estimator at any N . Their Theorem 4.2 for one covariate:

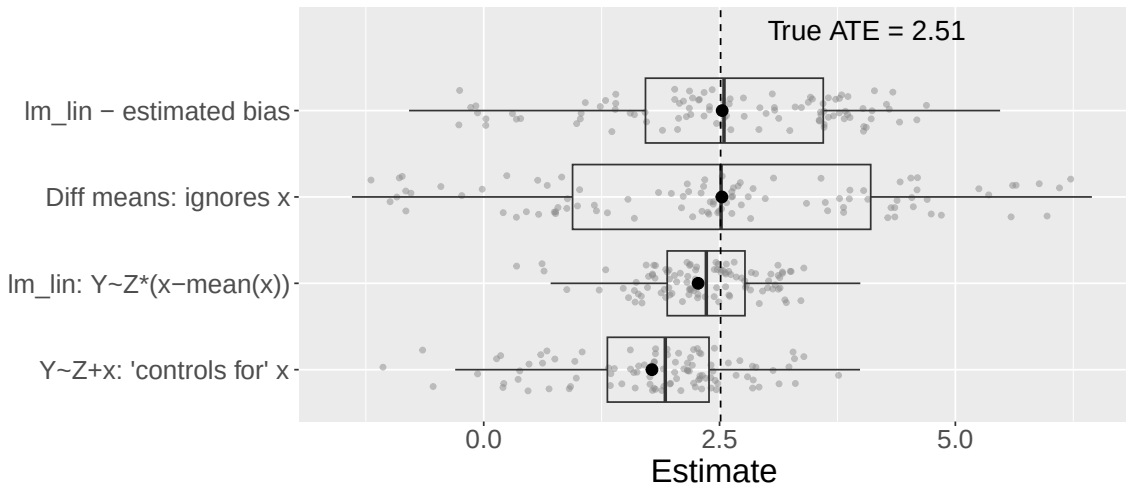
```
cma_bias_hat <- function(Y, Z, x) {  
  n <- length(Y)  
  xs <- (x - mean(x)) / sqrt(mean((x - mean(x))^2)) # standardize x  
  h <- xs^2 # the leverage of each unit  
  zt <- Z == 1  
  zc <- Z == 0  
  nT <- sum(zt)  
  nC <- sum(zc)  
  xbT <- mean(xs[zt])  
  xbC <- mean(xs[zc])  
  LT <- mean(xs[zt]^2) - xbT^2 # within-group variances of x ...  
  LC <- mean(xs[zc]^2) - xbC^2  
  NT <- mean(Y[zt] * xs[zt]) - mean(Y[zt]) * xbT # ... covariances of x, Y  
  NC <- mean(Y[zc] * xs[zc]) - mean(Y[zc]) * xbC  
  covhT <- mean(h[zt] * Y[zt]) - mean(h[zt]) * mean(Y[zt]) # leverage, Y  
  covhC <- mean(h[zc] * Y[zc]) - mean(h[zc]) * mean(Y[zc])  
  nT / ((n - 2) * nC) * covhC + xbC * (1 / LC - 1) * NC -  
    nC / ((n - 2) * nT) * covhT - xbT * (1 / LT - 1) * NT  
}
```

The fourth estimator: `lm_lin` minus its estimated bias

```
cma_est_fun <- function(data) {  
  lin_fit <- lm_lin(Y ~ Z, covariates = ~x, data = data)  
  res <- tidy(lin_fit) %>% filter(term == "Z")  
  res$estimate <- res$estimate - cma_bias_hat(data$Y, data$Z, data$x)  
  ## Keep the uncorrected estimator's standard error: the correction  
  ## does not change the large-sample distribution (Chang et al. 2024).  
  ## Watch what this does to coverage at N = 20 below  
  res$conf.low <- res$estimate - qt(.975, df = res$df) * res$std.error  
  res$conf.high <- res$estimate + qt(.975, df = res$df) * res$std.error  
  return(res)  
}  
  
est_c4 <- declare_estimator(  
  handler = label_estimator(cma_est_fun), inquiry = estimandATEc,  
  label = "lm_lin - estimated bias"  
)  
  
design_cov <- base_design_cov + estimandATEc +  
  est_c1 + est_c2 + est_c3 + est_c4
```

Then, as before, we reassign treatment 10,000 times and apply all four estimators to each simulated experiment.

Which estimator should we use?



Each gray point is the estimate from one possible experiment (100 of the 10,000); the black dot marks the average.

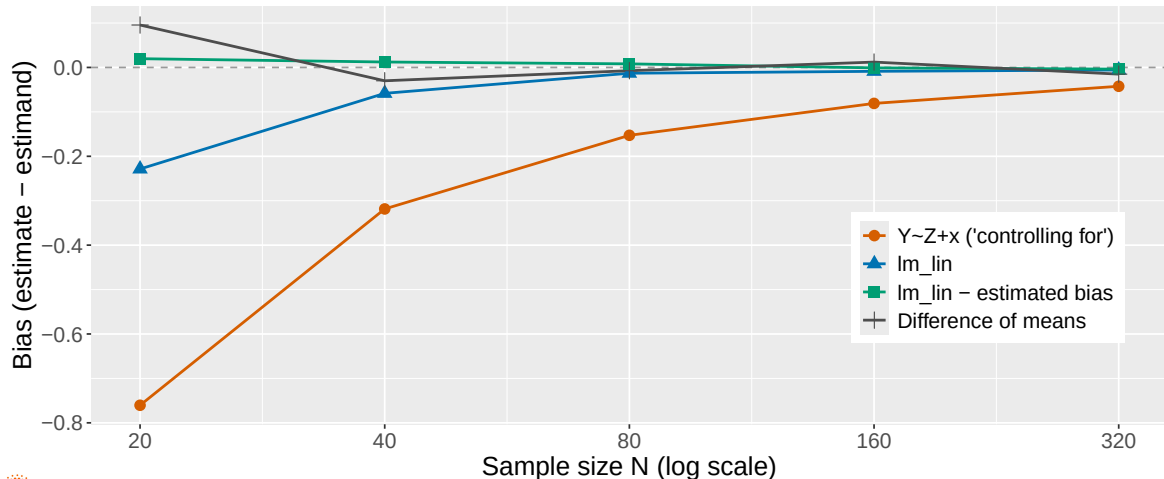
Which estimator is closer to the truth?

Estimator	Bias	SD Estimate	RMSE	Coverage
Diff means: ignores x	0.01	1.93	1.93	0.90
lm_lin - estimated bias	0.02	1.31	1.31	0.63
lm_lin: $Y \sim Z^*(x - \text{mean}(x))$	-0.24	0.73	0.77	0.88
$Y \sim Z + x$: 'controls for' x	-0.73	0.92	1.18	0.81

- $Y \sim Z + x$ underestimates the ATE of 2.51 by 0.73 — about 29% — and its intervals cover only 81% of the time.
- lm_lin has the smallest RMSE; its bias is about a third of the naive bias but not zero.
- The exact correction removes the bias — and pays for it with variance. Its borrowed standard error knows nothing about that extra noise, so its intervals cover only 63%. Unbiasedness is one operating characteristic, not the whole game.
- Even the unbiased difference of means covers 90%, not 95%: $N = 20$ with a skewed outcome strains every Normal-approximation interval here.

The adjustment bias fades as the experiment grows

Copies of the same 20 units, so only N changes: doubling the experiment roughly halves the naive bias. Unlike the adjusted logit coefficient above, this is a small-sample problem, not a wrong-estimand problem.



Advice about covariance adjustment

- In moderate and large experiments: adjust using a few **pre-specified**, prognostic covariates via `lm_lin()` — or, identically, the plug-in with an interacted linear model (Lin 2013). You gain precision; the remaining bias is of order $1/N$.
- In small experiments (tens of units): the bias can be a visible share of the effect and can distort coverage. Report the unadjusted difference in means alongside any adjusted estimate, and prefer to use covariates in the **design** (blocking) rather than in the analysis.
- An exactly unbiased correction exists (Chang et al. 2024). It trades bias for variance: at $N = 20$ the RMSE still favored `lm_lin`. Choose estimators by their operating characteristics — bias, precision, coverage — by simulating your own design, as throughout this session.
- Choose covariates before seeing outcomes. Adjusting after looking at results invites fishing (more on this in the transparency session).

Section 8

Conclusion

Final thoughts on the basics of estimation

- Counterfactual causal estimands are unobserved functions of potential outcomes.
- Estimators are recipes or computational formulas that use observed data to learn about an estimand.
- Good estimators produce estimates that are close to the true value of the estimand.
- (Connecting estimation with testing) Standard errors of estimators allow us to calculate confidence intervals and p -values. Certain estimators have larger or smaller (or more or less correct) standard errors.
- You can assess the utility of a chosen estimator for a given estimand by simulation.

Section 9

Causal effects that differ by groups or covariates

Effects that differ by groups I

If our theory suggests that effects should differ by group, how can we assess the evidence for or against such claims?

- We can **design** for an assessment of this theory by creating a block-randomized study — with blocks defined by the theoretically relevant groups.
- We can **plan** for such an assessment by (1) **pre-registering specific subgroup analyses** before the experiment is run and the data are collected (whether or not we block on that group in the design phase) and (2) making sure to measure group membership during baseline data collection pre-treatment

Effects that differ by groups II

- If we have not planned ahead, subgroup-specific analyses can be useful as explorations but should not be understood as confirmatory: testing too many hypotheses can inflate the false positive rate.
- We **should not use groups formed by treatment**. (This is either “mediation analysis” or “conditioning on post-treatment variables” and deserves its own module.)

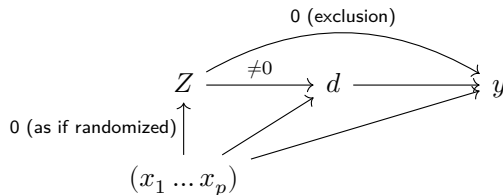
Section 10

Causal effects when we do not control the dose

Defining causal effects I

Imagine a door-to-door communication experiment where some houses are randomly assigned to receive a visit. Note that we now use Z and d instead of T .

- Z_i is random assignment to a visit ($Z_i = 1$) or not ($Z_i = 0$).
- $d_{i,Z_i=1} = 1$ means that person i would open the door to have a conversation when assigned a visit.
- $d_{i,Z_i=1} = 0$ means that person i would not open the door to have a conversation when assigned a visit.
- Opening the door is an outcome of the treatment.



Defining causal effects II

- $y_{i,Z_i=1,d_{i,Z_i=1}=1}$ is the potential outcome for people who were assigned a visit and who opened the door. (“Compliers” or “Always-takers”)
- $y_{i,0,d_{i,0}=1}$ is the potential outcome for people who were not assigned a visit and who open the door. (“Defiers” or “Always-takers”)
- $y_{i,0,d_{i,0}=0}$ is the potential outcome for people who were not assigned a visit and who do not open the door. (“Compliers” or “Never-takers”)

Defining causal effects III

We could also write $y_{i,Z_i=0,d_{i,Z_i=1}=1}$ for people who were not assigned a visit but who would have opened the door had they been assigned a visit, etc.

In this case, we can simplify our potential outcomes:

- $y_{i,0,d_{i,1}=1} = y_{i,0,d_{i,1}=0} = y_{i,0,d_{i,0}=0}$ because the outcome is the same whether or not the door is opened.

Defining causal effects IV

We can simplify the ways in which people get a dose of the treatment like so (where d is lower case reflecting the idea that whether someone opens the door when visited or not is a fixed attribute, like a potential outcome).

- Y : outcome ($y_{i,Z}$ or $y_{i,Z_i=1}$ for the potential outcome to treatment for person i , fixed)
- X : covariate/baseline variable
- Z : treatment assignment ($Z_i = 1$ if assigned to a visit, $Z_i = 0$ if not assigned to a visit)
- D : treatment received ($D_i = 1$ if person i opens the door, $D_i = 0$ if the person decides not to open the door) (using D here because $D_i = d_{i,1}Z_i + d_{i,0}(1 - Z_i)$)

Defining causal effects V

We have two causal effects of Z : $Z \rightarrow Y$ (δ , ITT, ITT_Y) and $Z \rightarrow D$ (GG call this ITT_D).

And different types of people can react differently to the attempt to move the dose with the instrument.

		$Z = 1$	
		$D = 0$	$D = 1$
$Z = 0$	$D = 0$	Never-takers	Compliers
	$D = 1$	Defiers	Always-takers

Defining causal effects VI

$$ITT = ITT_Y = \delta = \bar{y}_{Z=1} - \bar{y}_{Z=0}.$$

But, in this design, $\bar{y}_{Z=1} = \bar{y}_1$ is split into pieces: the outcome of those who opened the door (Compliers, Always-takers, and Defiers). We use p_C for the proportion of compliers in the study.

$$\bar{y}_1 = (\bar{y}_1|C)p_C + (\bar{y}_1|A)p_A + (\bar{y}_1|N)p_N + (\bar{y}_1|D)p_D. \quad (3)$$

And $\bar{y}_0$ is also split into pieces

$$\bar{y}_0 = (\bar{y}_0|C)p_C + (\bar{y}_0|A)p_A + (\bar{y}_0|N)p_N + (\bar{y}_0|D)p_D. \quad (4)$$

Defining causal effects VII

So, the ITT itself is a combination of the effects of Z on Y within these different groups (imagine substituting in and then re-arranging so that we have a set of ITTs, one for each type of subject). But we can still estimate the ITT because we have unbiased estimators of $\bar{y}_1$ and $\bar{y}_0$ within each type.

Learning about the ITT I

First, we can learn about the effect of the policy itself. To write down the ITT, we do not need to consider all of the types of subjects above. We have no defiers ($p_D = 0$) and we know that the ITT for both Always-takers and Never-takers is 0.

$$\bar{y}_1 = (\bar{y}_1|C)p_C + (\bar{y}_1|A)p_A + (\bar{y}_1|N)p_N \quad (5)$$

$$\bar{y}_0 = (\bar{y}_0|C)p_C + (\bar{y}_0|A)p_A + (\bar{y}_0|N)p_N \quad (6)$$

Learning about the ITT II

First, we can learn about the effect of the policy itself. To write down the ITT, we do not need to consider all of the types of subjects above. In this case, we have no defiers ($p_D = 0$) and we know that the ITT for both Always-takers and Never-takers is 0.

$$ITT = \bar{y}_1 - \bar{y}_0 \quad (7)$$

$$= ((\bar{y}_1|C)p_C + (\bar{y}_1|A)p_A + (\bar{y}_1|N)p_N) - \quad (8)$$

$$((\bar{y}_0|C)p_C + (\bar{y}_0|A)p_A + (\bar{y}_0|N)p_N) \quad (9)$$

Collecting each type together — to have an ITT for each type.

$$= ((\bar{y}_1|C)p_C - (\bar{y}_0|C)p_C) + ((\bar{y}_1|A)p_A - (\bar{y}_1|A)p_A) + \quad (10)$$

$$((\bar{y}_1|N)p_N - (\bar{y}_0|N)p_N) \quad (11)$$

$$= ((\bar{y}_1|C) - (\bar{y}_0|C))p_C + \quad (12)$$

$$((\bar{y}_1|A) - (\bar{y}_0|A))p_A + ((\bar{y}_1|N) - (\bar{y}_0|N))p_N \quad (13)$$

Learning about the ITT III

$$ITT = \bar{y}_1 - \bar{y}_0 \quad (14)$$

$$= ((\bar{y}_1|C)p_C + (\bar{y}_1|A)p_A + (\bar{y}_1|N)p_N) - \quad (15)$$

$$((\bar{y}_0|C)p_C + (\bar{y}_0|A)p_A + (\bar{y}_0|N)p_N) \quad (16)$$

$$= ((\bar{y}_1|C)p_C - (\bar{y}_0|C)p_C) + ((\bar{y}_1|A)p_A - (\bar{y}_0|A)p_A) + \quad (17)$$

$$((\bar{y}_1|N)p_N - (\bar{y}_0|N)p_N) \quad (18)$$

$$= ((\bar{y}_1|C) - (\bar{y}_0|C))p_C + ((\bar{y}_1|A) - (\bar{y}_0|A))p_A + \quad (19)$$

$$((\bar{y}_1|N) - (\bar{y}_0|N))p_N \quad (20)$$

Learning about the ITT IV

And if the effect of the dose can only occur for those who open the door, and only those assigned to treatment can open the door, then:

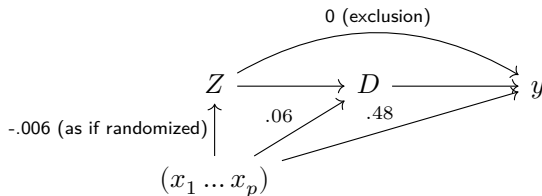
$$((\bar{y}_1|A) - (\bar{y}_0|A))p_A = 0 \text{ and } ((\bar{y}_1|N) - (\bar{y}_0|N))p_N = 0 \quad (21)$$

$$ITT = ((\bar{y}_1|C) - (\bar{y}_0|C))p_C = (CACE)p_C. \quad (22)$$

The complier average causal effect I

We would also like to learn about the causal effect of opening the door and having the conversation, the theoretically interesting effect.

But this comparison is confounded by x : a simple comparison $\bar{Y}|D = 1 - \bar{Y}|D = 0$ shows us differences in the outcome due to x in addition to the difference caused by D . (The numbers below come from a simulation)



The complier average causal effect II

```
with(dat, cor(Y, x)) ## can be any number  
with(dat, cor(d, x)) # can be any number  
with(dat, cor(Z, x)) ## should be near zero
```

But we just saw that in this design and with these assumptions (including the SUTVA assumption) $ITT = ((\bar{y}_1|C) - (\bar{y}_0|C))p_C = (CACE)p_C$, so we can define $CACE = ITT/p_C$.

How to calculate the ITT and CACE/LATE I

Some example data (where we know all of the potential outcomes):

X	u	type	Z	pZ	DZ1	YD0Z0	YD1Z0	YD0Z1	YD1Z1	D	Y
2	1.30	Complier	1	0	1	1.30	1.73	1.30	1.73	1	1.73
3	1.12	Complier	0	0	1	1.12	1.55	1.12	1.55	0	1.12

How to calculate the ITT and CACE/LATE II

The ITT and CACE (the parts)

```
itt_y <- difference_in_means(Y ~ Z, data = dat0)
itt_y
```

Design: Standard

	Estimate	Std. Error	t value	Pr(> t)	CI Lower	CI Upper	DF
Z	0.06457	0.1712	0.3772	0.7068	-0.2751	0.4043	97.84

```
itt_d <- difference_in_means(D ~ Z, data = dat0)
itt_d
```

Design: Standard

	Estimate	Std. Error	t value	Pr(> t)	CI Lower	CI Upper	DF
Z	0.68	0.07307	9.307	8.454e-15	0.5348	0.8252	89.31

How to calculate the ITT and CACE/LATE III

All together:¹

```
cace_est <- iv_robust(Y ~ D | Z, data = dat0)
cace_est
```

	Estimate	Std. Error	t value	Pr(> t)	CI Lower	CI Upper	DF
(Intercept)	0.39841	0.1372	2.9046	0.004545	0.1262	0.6706	98
D	0.09495	0.2514	0.3778	0.706412	-0.4038	0.5938	98

same as:

```
coef(itt_y)[["Z"]] / coef(itt_d)[["Z"]]
```

```
[1] 0.09495
```

¹works well when $Z \rightarrow D$ is not weak. See Imbens and Rosenbaum (2005) for a cautionary tale.

Summary of Encouragement/Complier/Dose oriented designs:

- Analyze as you randomized, even when you don't control the dose
- The danger of per-protocol analysis.

References

- Chang, Haoge, Joel A. Middleton, and P. M. Aronow. 2024. "Exact Bias Correction for Linear Adjustment of Randomized Controlled Trials." *Econometrica* 92 (5): 1503–19.
<https://doi.org/10.3982/ECTA20289>.
- Freedman, David A. 2008a. "On regression adjustments to experimental data." *Advances in Applied Mathematics* 40 (2): 180–93.
- Freedman, David A. 2008b. "Randomization Does Not Justify Logistic Regression." *Statistical Science* 23 (2): 237–49.
- Imbens, Guido W., and Paul R. Rosenbaum. 2005. "Robust, Accurate Confidence Intervals with a Weak Instrument: Quarter of Birth and Education." *Journal of the Royal Statistical Society Series A* 168 (1): 109–26.
- Lin, Winston. 2013. "Agnostic Notes on Regression Adjustments to Experimental Data: Reexamining Angrist and Pischke's Critique." *The Annals of Applied Statistics* 7 (1): 295–318.